

# Module 3.3 — Change Summary for Review

**File:** `3.3 Tuning Tree-Based Models.ipynb` (the plain, no-suffix lecture — the file we agreed to keep as canonical)

**Status:** Change applied, awaiting review. `3.3 Tuning Tree-Based Models Using f1.ipynb` has **not** been deleted yet — that happens only after this is approved.

## What changed

Only one cell was touched — the data-setup cell near the top of the notebook. Everything else (Decision Tree / Random Forest / XGBoost tuning, the early-stopping section, the final model-save cell) is untouched.

## Before

```

import numpy as np
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

import plotly.graph_objects as go
from plotly.subplots import make_subplots

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

from sklearn.model_selection import (GridSearchCV, RandomizedSearchCV,
                                     StratifiedKFold, cross_val_score)
from sklearn.metrics import roc_auc_score, make_scorer, f1_score
import time

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Load and prepare data (same as 3.2)
train_df = pd.read_csv(f'{project_path}{data_filepath}training.csv')
test_df = pd.read_csv(f'{project_path}{data_filepath}testing.csv')
train_df['DEPARTED'] = (train_df['SEM_3_STATUS'] != 'E').astype(int)
test_df['DEPARTED'] = (test_df['SEM_3_STATUS'] != 'E').astype(int)

numeric_features =
['HS_GPA', 'HS_MATH_GPA', 'HS_ENGL_GPA', 'UNITS_ATTEMPTED_1', 'UNITS_ATTEMPTED_2',
 'UNITS_COMPLETED_1', 'UNITS_COMPLETED_2', 'DFW_UNITS_1', 'DFW_UNITS_2', 'GPA_1',
  'DFW_RATE_1', 'DFW_RATE_2', 'GRADE_POINTS_1', 'GRADE_POINTS_2']
categorical_features =
['RACE_ETHNICITY', 'GENDER', 'FIRST_GEN_STATUS', 'COLLEGE']

train_enc = pd.get_dummies(train_df[numeric_features +
                                     categorical_features],
                           columns=categorical_features, drop_first=True)
test_enc = pd.get_dummies(test_df[numeric_features +
                                   categorical_features],
                          columns=categorical_features, drop_first=True)
train_enc, test_enc = train_enc.align(test_enc, join='left', axis=1,
                                     fill_value=0)
train_enc = train_enc.fillna(train_enc.median())

```

```
test_enc = test_enc.fillna(train_enc.median())

X_train, y_train = train_enc, train_df['DEPARTED']
X_test, y_test = test_enc, test_df['DEPARTED']

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
print(f"Data loaded: {X_train.shape[0]:,} training, {X_test.shape[0]:,}
testing samples")
```

## After

```

import numpy as np
import pandas as pd
import joblib, os
import warnings
warnings.filterwarnings('ignore')

import plotly.graph_objects as go
from plotly.subplots import make_subplots

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

from sklearn.model_selection import (GridSearchCV, RandomizedSearchCV,
                                     StratifiedKFold, cross_val_score)
from sklearn.metrics import make_scorer, f1_score
import time

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

ARTIFACT_DIR = f'{project_path}{course3_models}' # artifacts persist
here
os.makedirs(ARTIFACT_DIR, exist_ok=True)

# Load and prepare data (same as 3.2)
train_df = pd.read_csv(f'{project_path}{data_filepath}training.csv')
test_df = pd.read_csv(f'{project_path}{data_filepath}testing.csv')
train_df['DEPARTED'] = (train_df['SEM_3_STATUS'] != 'E').astype(int)
test_df['DEPARTED'] = (test_df['SEM_3_STATUS'] != 'E').astype(int)

numeric_features =
['HS_GPA', 'HS_MATH_GPA', 'HS_ENGL_GPA', 'UNITS_ATTEMPTED_1', 'UNITS_ATTEMPTED_2',
 'UNITS_COMPLETED_1', 'UNITS_COMPLETED_2', 'DFW_UNITS_1', 'DFW_UNITS_2', 'GPA_1',
 'DFW_RATE_1', 'DFW_RATE_2', 'GRADE_POINTS_1', 'GRADE_POINTS_2']
categorical_features =
['RACE_ETHNICITY', 'GENDER', 'FIRST_GEN_STATUS', 'COLLEGE']

train_enc = pd.get_dummies(train_df[numeric_features +
                                     categorical_features],
                           columns=categorical_features, drop_first=True)
test_enc = pd.get_dummies(test_df[numeric_features +

```

```

categorical_features],
                                columns=categorical_features, drop_first=True)
train_enc, test_enc = train_enc.align(test_enc, join='left', axis=1,
fill_value=0)

# --- Capture medians BEFORE filling so they persist; apply TRAIN medians
to both ---
train_medians = train_enc.median()
train_enc = train_enc.fillna(train_medians)
test_enc = test_enc.fillna(train_medians)

X_train, y_train = train_enc, train_df['DEPARTED']
X_test, y_test = test_enc, test_df['DEPARTED']

feature_columns = list(train_enc.columns)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
print(f"Data loaded: {X_train.shape[0]:,} training, {X_test.shape[0]:,}
testing samples")

# --- Persist artifacts for 3.4 ---
joblib.dump(feature_columns, f'{ARTIFACT_DIR}feature_columns.pkl')
joblib.dump(train_medians, f'{ARTIFACT_DIR}train_medians.pkl')
print(f"Saved {len(feature_columns)} feature columns +
{len(train_medians)} medians → {ARTIFACT_DIR}")

```

## What the two changes do

1. **Added the missing helper-file save** (`feature_columns.pkl`, `train_medians.pkl`). This is the piece `3.4 Evaluating Tree-Based Models.ipynb` needs that only the “Using f1” file used to produce. This notebook now saves it too, alongside the three tuned `.joblib` models it already saved — so a single run of this one notebook produces everything 3.4 needs.
2. **Removed the unused `roc_auc_score` import**. It was imported but never called anywhere in this file, so the file is now unambiguously F1-only.

## Why the plain file, not “Using f1”

Checked every AUC/F1 usage in both files line by line. Both tune on `scoring='f1'` — that part was already fine in both. The real difference:

| Model                    | "Using f1" reports                | plain file reports           |
|--------------------------|-----------------------------------|------------------------------|
| Decision Tree            | Best CV F1 + <b>Test AUC</b>      | Best CV F1 + <b>Test F1</b>  |
| Random Forest            | Best CV F1 + <b>Test AUC</b>      | Best CV F1 + <b>Test F1</b>  |
| XGBoost                  | Best CV F1 + <b>Test AUC</b>      | Best CV F1 + <b>Test F1</b>  |
| XGBoost (early stopping) | <b>Test AUC</b>                   | <b>Test F1</b>               |
| Final summary table      | Has a <code>ROC-AUC</code> column | F1 / Precision / Recall only |

Despite its name, "Using f1" tunes on F1 but reports the held-out **test-set** result in AUC every single time — six separate, real, executed `roc_auc_score` calls. The plain file is F1-consistent end to end: tunes on F1, reports test performance in F1, with zero AUC anywhere (until this edit removed even the one unused import).

## Verification status

No data files available locally yet, so this has been verified **statically only**:

- Every variable ( `feature_columns`, `train_medians`, `joblib`, `os`, `ARTIFACT_DIR` ) is defined before it's used.
- Zero remaining `roc_auc_score` references anywhere in the file.
- The saved filenames exactly match what `3.4 Evaluating Tree-Based Models.ipynb` already loads.

Not yet confirmed by actually running the notebook end-to-end against real data.

## Next step

Once this is approved, `3.3 Tuning Tree-Based Models Using f1.ipynb` will be removed so only one canonical 3.3 file remains.